

@codedsoul_05

Learn Today
Code Tomorrow
Conquer Forever ❤️

From
Basics to
Advanced
Concepts

THE COMPLETE DSA HANDBOOK

The best time
to plant a tree
was 20 years ago.
The second best
time is NOW.

≡ Master DSA. Crack Interviews. Build Real World Skills. ≡

What's Inside?

- ① Introduction to DSA
- ② Time & Space Complexity
- ③ Arrays
- ④ Strings & Hashing
- ⑤ Linked List, Stack, Queue & Deque
- ⑥ Trees & Binary Search Trees
- ⑦ Graphs
- ⑧ Searching & Sorting
- ⑨ Recursion, Backtracking & DP
- ⑩ Greedy, Heaps, Tries & Bit Manipulation

Perfect For :

- ✓ Beginners who want strong fundamentals
- ✓ Students for college exams & placements
- ✓ Anyone preparing for coding interviews
- ✓ Quick revision anytime, anywhere!

One swipe today,
Better tomorrow!



You Will Get :



Clear Concepts
with Examples



Important
Notes & Tips



Interview
Questions



Step-by-Step
Learning



Build Strong
Problem Solving

Ready to level up your DSA game?

SWIPE to begin >>>

CONSISTENT TODAY, SUCCESS TOMORROW. ❤️

Save this
for later! 📌



INTRODUCTION TO DSA



1. What is DSA ?

DSA stands for **Data Structures and Algorithms**.

It is a way of organizing, managing and storing data so that we can use it efficiently. It also includes a set of well-defined steps (algorithms) to solve problems and design efficient programs.

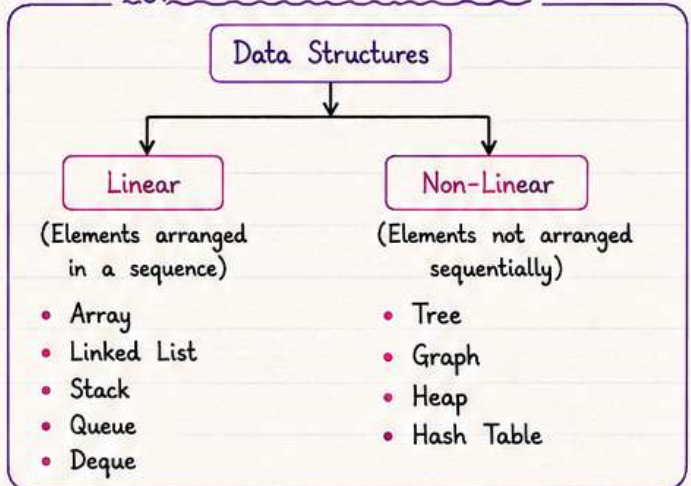
2. Why Learn DSA ?

- Helps in problem solving
- Improves code efficiency and optimization
- Essential for placements and interviews
- Builds strong programming foundation
- Helps in writing scalable and maintainable code
- Used in real-world applications

3. Data Structure vs Algorithm

Data Structure	Algorithm
• It is a way of storing and organizing data. • Focus on data and operations on data. • Example - Array, Linked List, Stack, Queue, Tree, Graph	• It is a set of step-by-step instructions to solve a problem. • Focus on logic and sequence of steps. • Example - Sorting, Searching, Dynamic Programming, Greedy Algorithm

4. Types of Data Structures



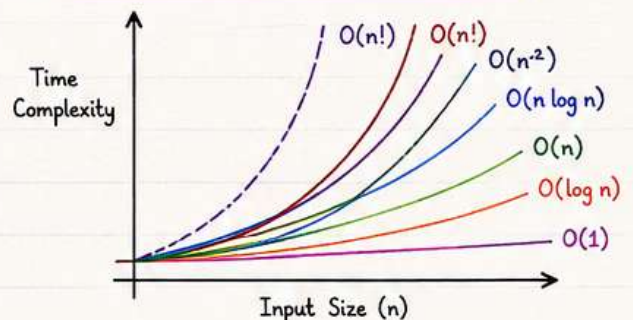
5. Big-O Notation (Time Complexity Overview)

Big-O notation is used to describe the upper bound of the growth of an algorithm's running time or space as the input size increases.

Notation	Name	Example	Growth
$O(1)$	Constant	Accessing array element	1
$O(\log n)$	Logarithmic	Binary Search	$\log n$
$O(n)$	Linear	Traversing array	n
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort	$n \log n$
$O(n^2)$	Quadratic	Bubble Sort, Selection Sort	n^2
$O(2^n)$	Exponential	Recursive Fibonacci	2^n
$O(n!)$	Factorial	Travelling Salesman Problem	$n!$

Growth Order (from best to worst)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$



6. How DSA Helps in Problem Solving ?

Understand the Problem



Choose Right Data Structure



Apply Suitable Algorithm



Optimize (Time & Space)



Efficient Solution



Practice DSA Daily, Think Better, Code Efficiently !





2. TIME & SPACE COMPLEXITY



Time and Space Complexity helps us analyze the efficiency of an algorithm. It tells us how **resources** (time or memory) grow as the input size (n) increases.

1. TIME COMPLEXITY

It refers to the amount of **time** an algorithm takes to complete as a function of input size (n). We calculate the number of **basic operations**.

Common Rules :

- Ignore constants.
- Focus on the highest order term.
- Different inputs $\rightarrow$ consider worst case.
- Drop lower order terms.
- n is large enough.

Formal Definition :

$O(g(n))$ is the set of functions $f(n)$ for which there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

2. SPACE COMPLEXITY

It refers to the amount of **memory** an algorithm uses as a function of input size (n). It includes:

- **Auxiliary Space** $\rightarrow$ Extra space used (variables, data structures, recursion stack)
- **Input Space** $\rightarrow$ Space used to store input data

Example

If an array of size n is created and a loop runs n times.

Time Complexity $\rightarrow O(n)$

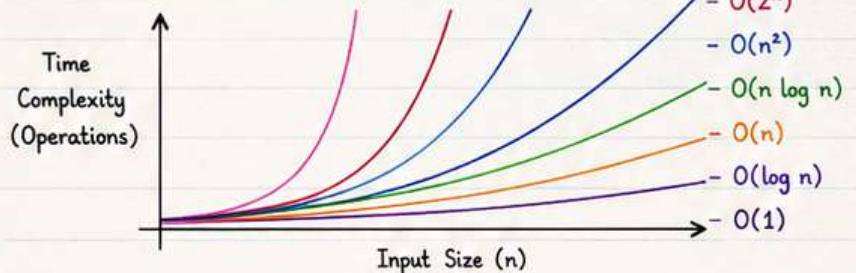
Space Complexity $\rightarrow O(n)$

3. BIG-O NOTATION

Big-O notation is used to describe the upper bound (worst case) of an algorithm's growth. It ignores machine specific details and focuses on scalability.

Growth Order (from best to worst)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$



4. COMMON TIME COMPLEXITIES

Notation	Name	Example	Explanation
$O(1)$	Constant	Accessing array element	Same time, no matter the input size
$O(\log n)$	Logarithmic	Binary Search	Reduces input by half at each step
$O(n)$	Linear	Traversing array	Time grows linearly with input size
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort	n times $\log n$
$O(n^2)$	Quadratic	Nested loops	Two nested loops
$O(2^n)$	Exponential	Recursive Fibonacci	Doubles with every increment
$O(n!)$	Factorial	TSP (Traveling Salesman)	Very very fast growth

5. SPACE COMPLEXITY EXAMPLES

Example	Space Complexity
Variables (int, char, etc.)	$O(1)$
Array of size n	$O(n)$
2D Array $n \times m$	$O(n \cdot m)$
Recursive function (depth = n)	$O(n)$
Binary Tree (n nodes)	$O(n)$

6. PROBLEM SOLVING APPROACH



Remember :

Always try to write optimized code. Evaluate your solution for worst case.

Practice Daily,
Code Efficiently!



Key Takeaway :

Good code works.
Optimized code scales!





3. ARRAYS



An Array is a linear data structure that stores elements of the same data type in contiguous (continuous) memory locations.

Why Arrays?

- Random access in $O(1)$
- Easy to traverse
- Cache friendly
- Forms the base for many other data structures

1. Basic Concept

- Elements are stored in adjacent memory locations.

Index :	0	1	2	3	4
Arr :	10	20	30	40	50
	↑				↑
	First Element				Last Element

- If base address of $\text{arr}[0] = B$ and size of each element = k bytes, then address of $\text{arr}[i] = B + i \times k$

2. Operations on Arrays

1. Access $\rightarrow O(1)$
2. Search $\rightarrow O(n)$
3. Insertion $\rightarrow O(n)$ (at end $\rightarrow O(1)$)
4. Deletion $\rightarrow O(n)$
5. Traversal $\rightarrow O(n)$



Note : Insertion or Deletion in the middle requires shifting of elements.

3. Types of Arrays

(A) By Dimension

- 1D Array (Linear Array)
- 2D Array (Matrix)
- Multi-Dimensional Array

(B) By Size

- Static Array $\rightarrow$ Size is fixed at compile time.
- Dynamic Array $\rightarrow$ Size can grow or shrink at runtime.

4. 2D Array (Matrix)

A 2D array is an array of arrays.

It can be visualized as a matrix with rows and columns.

Col $\rightarrow$	0	1	2
Row $\downarrow$			
0	1	2	3
1	4	5	6
2	7	8	9

- Number of rows = m , columns = n
- Total elements = $m \times n$
- Access element at (i, j) in $O(1)$

5. Memory Representation (1D Array)

Index :	0	1	2	3	4	5
Value :	5	15	25	35	45	55
Memory :	$[\text{addr}]$	$[\text{addr}+k]$	$[\text{addr}+2k]$	$[\text{addr}+3k]$	$[\text{addr}+4k]$	$[\text{addr}+5k]$

Formula :
Address of $\text{arr}[i] = \text{Base Address} + (i \times k)$ where,
 $k = \text{size of each element}$

6. Common Array Traversal (1D)

```
// Print all elements
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
```

Time Complexity
 $\rightarrow O(n)$

7. Common Interview Questions

- ① Find the largest/smallest element in an array.
- ② Check if an array is sorted.
- ③ Remove duplicates from a sorted array.
- ④ Left rotate the array by one place.
- ⑤ Move all zeroes to the end.
- ⑥ Union and Intersection of two arrays.
- ⑦ Find the missing number in an array of 1 to n .

8. Advantages vs Limitations

Advantages	Limitations
• Fast random access ($O(1)$) • Simple to implement • Cache friendly • Used as building block for other data structures	• Size is fixed in static arrays • Insertion/Deletion in middle is costly ($O(n)$) • Wastage of memory if size is over-estimated

Remember :

Choose the right approach, not the fastest code.

Practice Daily. Think Better. Code Efficiently.

Arrays are the foundation of DSA!



4. STRINGS & HASHING



A String is a sequence of characters. Hashing is a technique to map keys to values using a hash function for efficient storage and retrieval.

1. STRINGS

- A string is an array of characters terminated by '\0' (null character) in C/C++.
- In other languages, length is stored separately.

Index : 0 2 3 4 5
 Char :

H	e	l	b	\0
---	---	---	---	----

 String : "Hello" Length = 5

2. Common String Operations

- Length $\rightarrow O(n)$
- Access $\rightarrow O(1)$
- Search a character / substring $\rightarrow O(n)$
- Insert / Delete $\rightarrow O(n)$ (needs shifting)
- Concatenation $\rightarrow O(n)$
- Comparison $\rightarrow O(n)$

3. Common String Problems

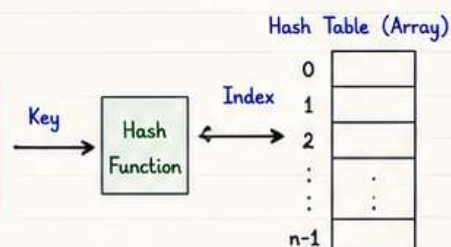
- Check Palindrome
- Reverse a String
- Check Anagram (same characters)
- Count Frequency of Characters
- Find First Non-Repeating Character
- Longest Substring Without Repeating Characters

4. STRING REPRESENTATION

- C / C++ $\rightarrow$ Character array (char str[])
- Java / Python $\rightarrow$ String objects
- Strings are **immutable** in Java and Python.

5. HASHING - INTRODUCTION

Hashing converts a key into an index of an array using a hash function. It helps in fast insertion, search and deletion.



6. HASH FUNCTIONS

A hash function $h(k)$ maps a key k to an index in range $[0, m-1]$ where m is the size of the hash table.

$$\text{Index} = h(k) = k \bmod m$$

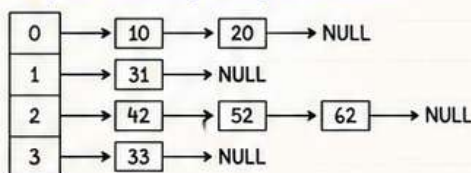
(Simple and commonly used)

7. COLLISIONS IN HASHING

When two different keys produce the same index.

Collision Handling Techniques :

(1) Separate Chaining (Using Linked List)



(2) Open Addressing (Probing)

- Linear Probing : $(i + 1) \% m$
- Quadratic Probing : $(i + j^2) \% m$
- Double Hashing : $(i + j * h_2(k)) \% m$

8. HASH TABLE OPERATIONS (Average Case)

Operation	Time Complexity
Search	$O(1)$
Insert	$O(1)$
Delete	$O(1)$

In worst case (when all keys collide) $\rightarrow O(n)$

9. HASHSET vs HASHMAP

HashSet (Set)	HashMap (Map / Dictionary)
- Stores only unique keys. - No duplicate values. - Implemented using HashMap internally. - Used to check presence of an element.	- Stores key-value pairs. - Keys are unique, values can be duplicate. - Each key maps to one value. - Used when we need to store and retrieve key-based data.

10. COMMON INTERVIEW QUESTIONS

- Implement a HashMap / HashSet.
- Find the first non-repeating character in a string.
- Check if two strings are anagrams.
- Group anagrams together.
- Count distinct elements in a window of size k .



Quick Tips

- Use strings for pattern matching, parsing, and text problems.
- Use hashing for fast lookup, counting frequencies, and avoiding duplicates.
- Always choose a good hash function and handle collisions efficiently.



Master Strings. Use Hashing. Solve Problems. Crack Interviews!



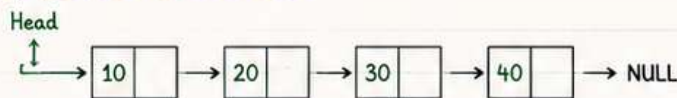
5. LINKED LISTS, STACK & QUEUE

These are linear data structures used to store collections of data.
Each has its own way of organizing and accessing elements.

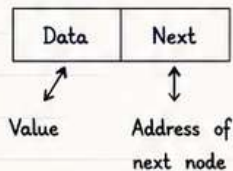
1. LINKED LIST

A linked list is a linear data structure where elements are not stored at contiguous memory locations. Each element (node) contains data and a reference (pointer) to the next node.

(A) SINGLY LINKED LIST



Node Structure :



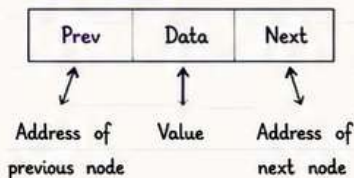
Common Operations

- Traversal $\rightarrow O(n)$
- Insertion $\rightarrow O(1)$ (at head)
- Deletion $\rightarrow O(1)$ (at head)

(B) DOUBLY LINKED LIST



Node Structure :



Use Cases

- Implement Stacks
- Implement Queues
- Undo/Redo operations
- Browser History
- Music Playlist

2. STACK (LIFO)

Stack follows Last In First Out principle.
Insertion and Deletion happen from the same end called TOP.



Operations

- Push (Insert) $\rightarrow O(1)$
- Pop (Delete) $\rightarrow O(1)$
- Peek (Top Element) $\rightarrow O(1)$
- isEmpty() $\rightarrow O(1)$

Applications

- Function Call Management (Recursion)
- Undo Operations
- Expression Evaluation
- Backtracking, DFS

3. QUEUE (FIFO)

Queue follows First In First Out principle.
Insertion happens at REAR and Deletion happens at FRONT.



Operations

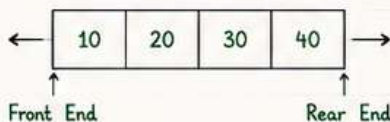
- Enqueue (Insert) $\rightarrow O(1)$
- Dequeue (Delete) $\rightarrow O(1)$
- Front() $\rightarrow O(1)$
- isEmpty() $\rightarrow O(1)$

Applications

- CPU Scheduling
- Printer Queue
- Level Order Traversal (Tree)
- Real-life Waiting Lines

4. DEQUE (DOUBLE ENDED QUEUE)

Deque allows insertion and deletion from both ends.



Operations

- Insert Front $\rightarrow O(1)$
- Insert Rear $\rightarrow O(1)$
- Delete Front $\rightarrow O(1)$
- Delete Rear $\rightarrow O(1)$

5. COMPARISON TABLE

Data Structure	Order	Insertion	Deletion	Access
Stack	LIFO	Top	Top	Top Element
Queue	FIFO	Rear	Front	Front Element
Deque	Double Ended	Both Ends	Both Ends	Both Ends
Linked List	Sequential	Any Position	Any Position	Sequential

★ KEY TAKEAWAY

- ✓ Linked List $\rightarrow$ Flexible size, not stored in contiguous memory.
- ✓ Stack $\rightarrow$ LIFO, great for recursion & backtracking.
- ✓ Queue $\rightarrow$ FIFO, used in scheduling & real-world waiting.
- ✓ Deque $\rightarrow$ Insertion & deletion from both ends.

Remember :

- Choose the right data structure based on the problem requirement.
- Understand the operations and their complexities.
- Practice Problems Daily to master these concepts!



Code it. Debug it. Optimize it. That's the DSA way!





6. TREES & BINARY SEARCH TREES



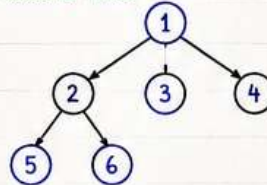
A Tree is a **hierarchical** data structure consisting of nodes.
It is widely used to represent hierarchical relationships.

1. TERMINOLOGY

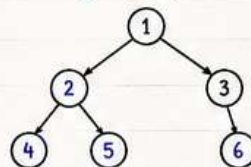
- Node** : Each element in a tree.
- Root** : The topmost node.
- Edge** : Connection between two nodes.
- Parent** : A node that has children.
- Child** : A node which is directly connected to another node when moving away from the root.
- Sibling** : Nodes that have the same parent.
- Leaf Node** : A node with no children.
- Degree** : Number of children of a node.
- Height** : Longest path from node to a leaf.
- Level** : Distance of node from root (root is at level 0).

2. TYPES OF TREES

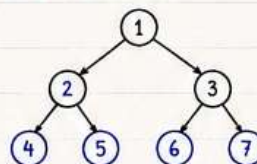
1. General Tree



2. Binary Tree (Max 2 children)



3. Full Binary Tree (Every node has 0 or 2 children)

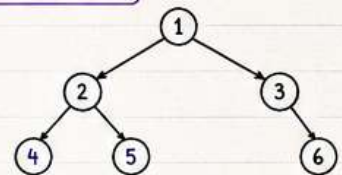


3. TREE TRAVERSALS

(A) Depth First Search (DFS)

- Preorder** (Root → Left → Right)
- Inorder** (Left → Root → Right)
- Postorder** (Left → Right → Root)

Example Tree



Traversal Orders

- Preorder** : 1 2 4 5 3 6
- Inorder** : 4 2 5 1 3 6
- Postorder** : 4 5 2 6 3 1

(B) Breadth First Search (BFS)

Level Order Traversal

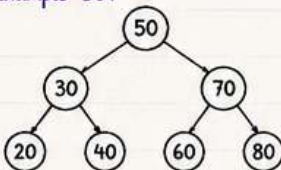
→ 1 2 3 4 5 6
(Use Queue)

4. BINARY SEARCH TREE (BST)

A BST is a binary tree where, for every node:

- All values in left subtree < node value
- All values in right subtree > node value
- Both left and right subtrees are also BSTs

Example BST



Properties

- Inorder traversal of BST gives sorted order.
- No duplicate values (usually).
- Efficient Search, Insert and Delete → $O(h)$ where h is height of tree.

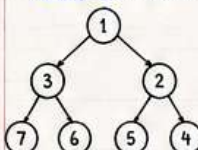
6. HEAP (INTRODUCTION)

A Heap is a complete binary tree that satisfies Heap property.

Types :

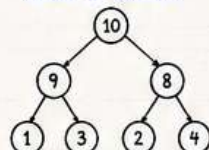
(A) Min Heap

Parent ≤ Children



(B) Max Heap

Parent ≥ Children



Applications

- Priority Queues
- Heap Sort
- Dijkstra's Algorithm

5. BASIC OPERATIONS IN BST

Operation	Description	Time Complexity
Search	Find a value in BST	$O(h)$
Insert	Insert a new value	$O(h)$
Delete	Remove a value	$O(h)$

Note :

- If tree is balanced → $h \approx \log n \rightarrow O(\log n)$
- If tree is skewed → $h \approx n \rightarrow O(n)$



7. TREE VS OTHER DATA STRUCTURES

Feature	Array	Linked List	Tree
Structure	Linear	Linear	Hierarchical
Memory	Contiguous	Non-contiguous	Non-contiguous
Access	$O(1)$ by index	$O(n)$	$O(h)$
Search	$O(n)$	$O(n)$	$O(h)$
Insertion	$O(n)$	$O(1)$	$O(h)$
Best Use	Simple data	Frequent insert/delete	Hierarchical data



KEY TAKEAWAYS

- ✓ Trees are hierarchical and non-linear.
- ✓ DFS → Use Stack (or Recursion)
- ✓ BFS → Use Queue
- ✓ BST → Left < Node < Right
- ✓ Balanced tree gives best performance.

Visualize the tree.
Understand the flow.
Crack the interview!



COMMON INTERVIEW QUESTIONS

- Implement all Tree Traversals.
- Check if a tree is BST.
- Find height / diameter of a tree.
- Level Order Traversal.
- Lowest Common Ancestor (LCA).



Trees make complex relationships simple. Master Trees, Master DSA! ❤️



7. GRAPHS



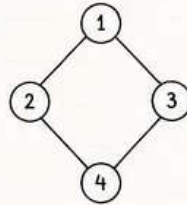
A Graph is a non-linear data structure consisting of **vertices (nodes)** and **edges** that connect pairs of vertices.

1. TERMINOLOGY

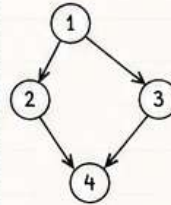
- **Vertex (Node)** : An entity in a graph.
- **Edge** : Connection between two vertices.
- **Undirected Graph** : Edges have no direction.
- **Directed Graph (Digraph)** : Edges have direction.
- **Weighted Graph** : Edges have weights/costs.
- **Path** : Sequence of vertices connected by edges.
- **Cycle** : Path that starts and ends at same vertex.
- **Degree** : Number of edges connected to a vertex.
 - In Undirected graph $\rightarrow \text{degree}(v)$
 - In Directed graph $\rightarrow \text{in-degree, out-degree}$

2. TYPES OF GRAPHS

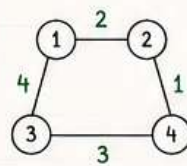
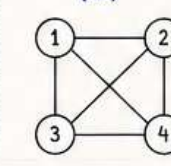
(A) Undirected Graph



(B) Directed Graph



(C) Weighted Graph

(D) Complete Graph (K_4)

3. REPRESENTATION

(A) Adjacency Matrix

	0	1	2	3
0	0	1	1	0
1	1	0	1	1
2	1	1	0	1
3	0	1	1	0

Space : $O(V^2)$
Check Edge : $O(1)$
Best for dense graphs

(B) Adjacency List

0 $\rightarrow$ 1 $\rightarrow$ 2
1 $\rightarrow$ 0 $\rightarrow$ 2 $\rightarrow$ 3
2 $\rightarrow$ 0 $\rightarrow$ 1 $\rightarrow$ 3
3 $\rightarrow$ 1 $\rightarrow$ 2

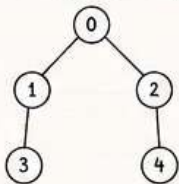
Space : $O(V + E)$
Iterate Neighbors : $O(\text{deg}(v))$
Best for sparse graphs

4. GRAPH TRAVERSALS

(A) BFS (Breadth First Search)

- Explores level by level.
- Uses Queue.
- Time Complexity : $O(V + E)$

Example (Start from 0)

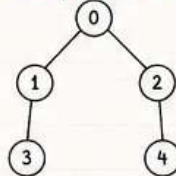


BFS Order :
0 1 2 3 4

(B) DFS (Depth First Search)

- Explores as deep as possible then backtracks.
- Uses Stack (or Recursion).
- Time Complexity : $O(V + E)$

Example (Start from 0)



DFS Order :
0 1 3 2 4

When to use?

- BFS $\rightarrow$ Shortest path in unweighted graph
- DFS $\rightarrow$ Topological Sort, Cycle Detection, Connected Components

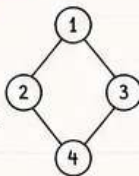
Common Applications

- ✓ Social Networks (Connections)
- ✓ Maps & Navigation (Shortest Path)
- ✓ Dependency Resolution (Topological Sort)
- ✓ Network Routing
- ✓ AI & Game State Space

5. CYCLE DETECTION

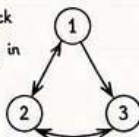
(A) Undirected Graph

- Use DFS / BFS
- If we visit a visited node that is not parent $\rightarrow$ Cycle



(B) Directed Graph

- Use DFS + Recursion Stack
- If we visit a node already in recursion stack $\rightarrow$ Cycle



6. SHORTEST PATH ALGORITHMS

(A) Unweighted Graph $\rightarrow$ BFS

- Finds shortest path (in terms of edges)
- Time Complexity : $O(V + E)$

(B) Weighted Graph (Non-negative)

Dijkstra's Algorithm

- Uses Min-Heap (Priority Queue)
- Time Complexity : $O((V + E) \log V)$



(C) Weighted Graph (With Negative Edges)

Bellman-Ford Algorithm

- Time Complexity : $O(V \times E)$
- Can detect negative weight cycle

7. GRAPH PROPERTIES

- In undirected graph, sum of all degrees = $2E$
- In directed graph, sum of in-degrees = E and sum of out-degrees = E
- A connected graph has a path between every pair of vertices.

8. BFS vs DFS

Feature	BFS	DFS
Strategy	Level by Level	Go Deep First
Data Structure	Queue	Stack / Recursion
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$
Best Use	Shortest Path	Topological Sort, Cycle Detection

9. COMMON INTERVIEW QUESTIONS

- Implement BFS and DFS.
- Detect cycle in undirected graph.
- Detect cycle in directed graph.
- Find number of connected components.
- Find shortest path in unweighted graph.
- Find shortest path in weighted graph.
- Topological Sort of a DAG.
- Check if graph is Bipartite.
- Bridge in a graph.
- Articulation Point in a graph.



KEY TAKEAWAYS



- ✓ Graphs model real-world relationships.
- ✓ BFS for shortest path in unweighted graphs.
- ✓ DFS for deep exploration and backtracking.
- ✓ Choose right representation based on density.
- ✓ Master traversal + cycle detection for interviews.



Understand the Graph. Traverse with Confidence. Solve with Logic.
Keep Practicing, Keep Growing! ❤️



8. SEARCHING & SORTING

Searching is the process of finding an element in a collection.

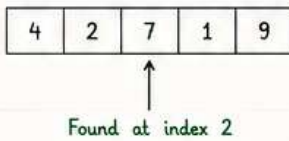
Sorting is the process of arranging elements in a particular order.

1. SEARCHING ALGORITHMS

(A) Linear Search

- Check each element one by one.
- Time Complexity : $O(n)$
- Space Complexity : $O(1)$

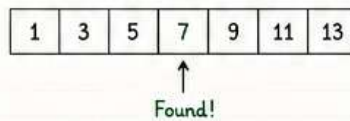
Example : arr = [4, 2, 7, 1, 9], key = 7



(B) Binary Search (For Sorted Array)

- Compare middle element.
- If key is smaller → search left half.
- If key is larger → search right half.
- Time Complexity : $O(\log n)$
- Space Complexity : $O(1)$ [Iterative] or $O(\log n)$ [Recursive]

Example : arr = [1, 3, 5, 7, 9, 11, 13], key = 7



2. COMPARISON OF SEARCHING

Algorithm	Best Case	Avg Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)^*$

* Iterative → $O(1)$, Recursive → $O(\log n)$

When to use?

- Linear Search → Unsorted/List data, small datasets.
- Binary Search → Sorted arrays, large datasets.

3. SORTING ALGORITHMS

(A) Bubble Sort

- Repeatedly swap adjacent elements if they are in wrong order.
- Time Complexity :
Best : $O(n)$
Avg / Worst : $O(n^2)$
- Space Complexity : $O(1)$
- Stable : Yes

Example : [5, 1, 4, 2]

Pass 1 : 1 4 2 5

Pass 2 : 1 2 4 5

Pass 3 : 1 2 4 5 ✓

(B) Selection Sort

- Find minimum element and place it at correct position.
- Time Complexity :
Best / Avg / Worst : $O(n^2)$
- Space Complexity : $O(1)$
- Stable : No

Example : [5, 1, 4, 2]

Pass 1 : 1 5 4 2

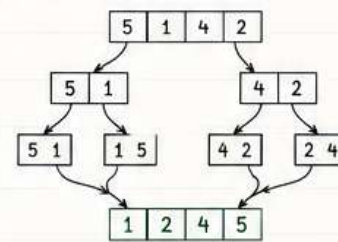
Pass 2 : 1 2 4 5

Pass 3 : 1 2 4 5 ✓

(C) Insertion Sort

- Insert current element in its correct position in the sorted part.
- Time Complexity :
Best : $O(n)$ (already sorted)
Avg / Worst : $O(n^2)$
- Space Complexity : $O(1)$
- Stable : Yes

Example : [5, 1, 4, 2]



(E) Quick Sort

- Choose a pivot, place it at correct position and sort left and right parts.
- Time Complexity :
Best / Avg : $O(n \log n)$
Worst : $O(n^2)$
- Space Complexity : $O(\log n)$ (Recursion Stack)
- Stable : No

Example : [5, 1, 4, 2]

Pivot = 5 → [1 4 2] 5 []

Pivot = 2 → [] 2 [4]

Combine → 1 2 4 5 ✓

4. COMPARISON OF SORTING ALGORITHMS

Algorithm	Best Case	Avg Case	Worst Case	Space	Stable
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No

5. IMPORTANT POINTS

- ★ Sorting is useful for optimization and faster searching.
- ★ Merge Sort is preferred for linked lists & large datasets.
- ★ Quick Sort is fastest in practice but worst case is $O(n^2)$.
- ★ Binary Search requires sorted data.
- ★ Always choose algorithm based on constraints.

6. COMMON INTERVIEW QUESTIONS

- Implement Binary Search.
- Why is Binary Search better than Linear Search?
- Sort an array of 0s, 1s and 2s.
- Find K^{th} largest / smallest element.
- Explain time complexity of different sorting algorithms.
- When will Quick Sort perform poorly?
- How does Merge Sort work? (Dry run)
- Which sorting algorithm is stable?

Practice these problems regularly to improve speed & accuracy!



Good Algorithms + Practice = Great Developer

Keep Coding, Keep Growing! ❤️



★ 9. RECURSION, BACKTRACKING & DYNAMIC PROGRAMMING ★

These are powerful techniques used to break down complex problems into simpler subproblems and build efficient solutions.

1. RECURSION

Recursion is a technique where a function calls itself to solve a smaller instance of the same problem.

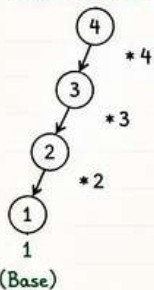
Key Points :

- Has a **Base Case** (stops recursion)
- Has **Recursive Case** (smaller problem)
- Uses **Call Stack** implicitly

Example : Factorial of n

```
fact(n):
    if n == 0 or n == 1:
        return 1      # Base Case
    return n * fact(n-1) # Recursive Case
```

Recursion Tree for $\text{fact}(4)$



Time & Space Complexity

- Time : $O(n)$
- Space : $O(n)$ (Recursion Stack)

4. RECURSION vs ITERATION

Recursion	Iteration
Uses Call Stack	No Call Stack
Higher memory	Lower memory
Easy to write	Slightly harder
May cause Stack Overflow	No Stack Overflow
Good for tree/graph	Good for simple loops

2. BACKTRACKING

Backtracking is an algorithmic technique for finding all (or some) solutions by trying candidates and abandoning ("backtracking") when a candidate does not lead to a valid solution.

Approach :

- ① Choose a candidate
- ② Explore further
- ③ If not valid → backtrack
- ④ Repeat

Example : N-Queens ($N = 4$)

1	Q			
2			Q	
3		Q		
4				Q
	1	2	3	4

Valid Solutions for $N = 4 \rightarrow 2$

Common Problems

- N-Queens
- Sudoku Solver
- Rat in a Maze
- Subset Sum
- Permutations

3. DYNAMIC PROGRAMMING (DP)

DP solves problems by breaking them into overlapping subproblems and storing the results to avoid recomputation.

Principles :

- Overlapping Subproblems
- Optimal Substructure
- Store results (Memoization / Tabulation)

Example : Fibonacci (n)

(A) Naive Recursion
 $T(n) = T(n-1) + T(n-2)$
 Exponential Time

(B) DP (Memoization)
 Store previously computed values in an array.
 Time : $O(n)$
 Space : $O(n)$

DP Approaches

(A) Memoization (Top-Down)

- Use Recursion
- Store results in array
- Easy to code
- Extra space ($O(n)$)

(B) Tabulation (Bottom-Up)

- Use Iteration
- Build solution bottom-up
- Usually better space
- More efficient

5. BACKTRACKING TEMPLATE

```
void solve(...){
    if (isSolution()){
        saveSolution();
        return;
    }
    for (each choice){
        if (isValid(choice)){
            makeChoice(choice);
            solve(...); // Recurse
            undoChoice(choice); // Backtrack
        }
    }
}
```

6. DP PROBLEM SOLVING STEPS

- ① Define the state (What is $dp[i]$ or $dp[i][j]$?)
- ② Write the recurrence relation
- ③ Choose approach (Memoization or Tabulation)
- ④ Initialize base cases
- ⑤ Compute answer
- ⑥ Analyze Time & Space Complexity

★ Not every problem is DP.
 Check for overlapping subproblems and optimal substructure first!



7. COMMON DP PROBLEMS

Problem	Description	Approach	Time Complexity	Space Complexity
Fibonacci	Find n^{th} Fibonacci number	1D DP	$O(n)$	$O(1) / O(n)$
Climbing Stairs	Ways to reach n^{th} stair	1D DP	$O(n)$	$O(1)$
0/1 Knapsack	Max value with weight limit	2D DP	$O(nW)$	$O(nW)$
Unbounded Knapsack	Infinite supply of items	2D / 1D DP	$O(nW)$	$O(W)$
Longest Common Subsequence	LCS of two strings	2D DP	$O(mn)$	$O(mn)$
Longest Increasing Subsequence	Length of LIS	1D DP (Binary Search)	$O(n \log n)$	$O(n)$
Matrix Chain Multiplication	Min cost to multiply matrices	2D DP	$O(n^3)$	$O(n^2)$

8. COMPLEXITY SUMMARY

Technique	Best Used For
Recursion	Problems with recursive structure
Backtracking	Finding all solutions / constraint satisfaction
DP	Optimization problems with overlapping subproblems

Practice is the key to master these techniques. Solve more problems and build pattern recognition! 😊

9. COMMON INTERVIEW QUESTIONS

- ① Write factorial using recursion.
- ② What is the base case in recursion?
- ③ Write fibonacci using memoization.
- ④ Solve N-Queens using backtracking.
- ⑤ What is the difference between backtracking and DP?
- ⑥ 0/1 Knapsack problem.
- ⑦ Longest Common Subsequence.
- ⑧ How does memoization work?
- ⑨ When to use recursion vs iteration?
- ⑩ Why is DP better than recursion for overlapping subproblems?

★ KEY TAKEAWAYS

- ✓ Recursion → Break into smaller instances
- ✓ Backtracking → Try & explore, backtrack when needed
- ✓ DP → Solve once, use many times
- ✓ Identify patterns → Solve smarter!



Think Recursively. Backtrack Smartly. Solve Optimally. Master Concepts. Crack Interviews.



★ 10. GREEDY, HEAPS, TRIES & BIT MANIPULATION ★

These techniques help in solving optimization, priority based, string related and low-level problems efficiently.

1. GREEDY ALGORITHM

- Make the locally optimal choice at every step with the hope of finding a global optimum.
- Works when problem has Greedy Choice Property + Optimal Substructure.

Example : Activity Selection

Activity	Start	End
A1	1	3
A2	2	5
A3	4	6
A4	6	7
A5	5	8
A6	8	9

Sorted by end time:
A1, A3, A4, A6

Max activities = 4

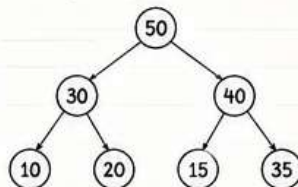
Common Problems

- Activity Selection
- Fractional Knapsack
- Huffman Coding
- Job Sequencing with Deadlines
- Minimum Spanning Tree

2. HEAPS (PRIORITY QUEUE)

- A complete binary tree.
- For Max Heap : parent $\geq$ children
- For Min Heap : parent $\leq$ children

Max Heap Example



Operations (All in $O(\log n)$)

- insert(x) $\rightarrow$ push
- extractMax() / extractMin() $\rightarrow$ pop
- peek() $\rightarrow$ get top
- heapify() $\rightarrow$ build heap

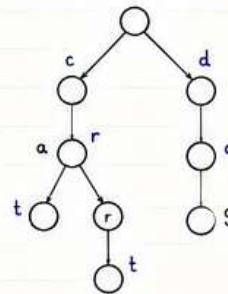
Applications

- ✓ Priority Queue
- ✓ Heap Sort
- ✓ Dijkstra's Algorithm
- ✓ Top K Elements

3. TRIES (PREFIX TREE)

- Tree like data structure for strings.
- Each node represents a character.
- Fast search of words with common prefixes.

Example : Insert words {cat, car, cart, dog}



Operations

- insert(word) $\rightarrow O(L)$
- search(word) $\rightarrow O(L)$
- startsWith(prefix) $\rightarrow O(L)$ (L = length of string)

Applications

- ✓ Autocomplete
- ✓ Spell Checker
- ✓ IP Routing (Longest Prefix Match)

4. BIT MANIPULATION

- Performs operations at bit level.
- Useful for optimization and low level tasks.

Basic Operations

x & y	AND	5 & 3 = 1
x y	OR	5 3 = 7
x ^ y	XOR	5 ^ 3 = 6
~x	NOT	~5 = -6
x << k	Left Shift	5 << 1 = 10
x >> k	Right Shift	5 >> 1 = 2

Common Tricks

- Check if number is even: $(n \& 1) == 0$
- Check if number is power of 2: $n > 0$ and $(n \& (n-1)) == 0$
- Set kth bit: $n | (1 \ll k)$
- Clear kth bit: $n \& ~(1 \ll k)$
- Toggle kth bit: $n ^ (1 \ll k)$
- Count set bits: use $(n \& (n-1))$ trick

5. COMPARISON TABLE

Technique	Best Used For	Time Complexity	Space
Greedy	Optimization Problems	Depends	Usually $O(1) - O(n)$
Heaps	Priority Based Problems	$O(\log n)$ per op	$O(n)$
Tries	String / Prefix Problems	$O(L)$ per op	$O(\text{ALPHABET} * N * L)$
Bit Manipulation	(N = number of strings, L = average length, ALPHABET = size of character set) Low level / Optimization	$O(1)$	$O(1)$

6. HEAP SORT (Using Max Heap)

- Build Max Heap $\rightarrow O(n)$
- Repeatedly extract max and place at end.

Time Complexity: $O(n \log n)$

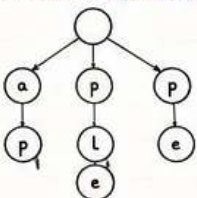
Space Complexity: $O(1)$

Example :



7. TRIE EXAMPLE WALKTHROUGH

Insert words: {app, apple, ape}



Search Operations

- search("app") $\rightarrow$ Found ✓
- search("appl") $\rightarrow$ Not Found ✗
- startsWith("ap") $\rightarrow$ True ✓
- startsWith("aq") $\rightarrow$ False ✗

Complexity

- insert $\rightarrow O(L)$
- search $\rightarrow O(L)$
- startsWith $\rightarrow O(L)$

8. COMMON PROBLEMS BY TOPIC

- Greedy** : Activity Selection, Fractional Knapsack, Huffman Coding, Job Sequencing, Minimum Spanning Tree
- Heaps** : Kth Largest Element, Merge k Sorted Lists, Heap Sort, Implement Priority Queue
- Tries** : Implement Trie, Word Search II, Longest Common Prefix
- Bit Manipulation** : Single Number, Count Set Bits, Power of Two, Missing Number, Subsets (using bits)

9. COMMON INTERVIEW QUESTIONS

- Explain Greedy Algorithm with an example.
- When Greedy fails? Give an example.
- Implement Min Heap. What are its operations?
- How does Heap Sort work?
- Implement Trie (insert, search, startsWith).
- What is the time complexity of Trie operations?
- Explain bitwise XOR with use cases.
- Check if a number is power of two without loops.
- Find the missing number in an array using XOR.
- How would you count set bits in a large number?

★ KEY TAKEAWAYS

- ✓ Greedy $\rightarrow$ Make best choice at each step.
- ✓ Heaps $\rightarrow$ Efficient for priority based operations.
- ✓ Tries $\rightarrow$ Great for string and prefix problems.
- ✓ Bit Manipulation $\rightarrow$ Optimize using bits, not loops.



Good Problems Build Skills. Smart Techniques Build Solutions.
Practice. Analyze. Optimize. That's the DSA Way!

